

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

Ордена Трудового Красного Знамени

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Московский технический университет связи и информатики»

Кафедра «Математическая кибернетика и информационные технологии»

Отчет по лабораторной работе №6

по дисциплине «Информационные технологии и программирование»

Выполнил: студент группы

БПИ2401

Старков Дмитрий Константинович

Проверил:

Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы	2
Ход работы.....	2
Вывод.....	7

Цель работы

Освоить коллекции в языке Java.

Ход работы

Начнем с выполнения первого задания:

```
1  import java.io.*;
2  import java.util.*;
3
4  public class TopWords {
5
6      public static void main(String[] args) {
7          File file = new File("text.txt");
8
9          Scanner scanner = null;
10         try {
11             scanner = new Scanner(file);
12         } catch (FileNotFoundException e) {
13             System.out.println("File not found.");
14             return;
15         }
16
17         Map<String, Integer> wordCount = new HashMap<>();
18
19         while (scanner.hasNext()) {
20             String word = scanner.next().toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
21             if (word.isEmpty()) {
22                 continue;
23             }
24
25             wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
26         }
27
28         scanner.close();
29
30         List<Map.Entry<String, Integer>> list = new ArrayList<>(wordCount.entrySet());
31
32         list.sort((o1, o2) -> o2.getValue() - o1.getValue());
33
34         int limit = Math.min(10, list.size());
35         for (int i = 0; i < limit; i++) {
36             Map.Entry<String, Integer> entry = list.get(i);
37             System.out.println((i + 1) + ". " + entry.getKey() + " - " + entry.getValue());
38         }
39     }
40 }
```

В начале я создаю переменную файла и подгружаю ее в сканнер. Далее идет создания словаря с типами ключ-значение равными String-Integer, соответственно для слова и его повторений в тексте. За основу для

реализации я взял именно HashMap, так как сортировку надо будет проводить по количеству повторений в тексте, а не по алфавитному порядку, поэтому на моменте со словарём порядок слов мне не важен.

Дальше идет обработка каждого слова с предварительной подготовки их к одному типу – строчные буквы и удаление любых символов, кроме латинских букв и арабских цифр. Если слово не является пустой строкой, то в словарь по ключу, являющимся этим словом, помещается либо значение счетчика для этого слова, либо 0, если это слово попадает впервые, после чего к полученному значению прибавляется единица.

Затем идёт подготовка к сортировке слов по их популярности, для этого я создаю список из элементов entrySet (ключ, значение). А так как мне нужно его отсортировать – я использую ArrayList, поддерживающий сортировку.

Для самой сортировки я использую лямбда-функцию, в которой вычитаю из второго значение первое, таким образом список сортируется по убыванию. Затем создаю переменную limit на случай, если в файле не наберется 10 слов, после чего запускаю цикл, который выводит каждую запись в новой строке.

Теперь второе задание

```
1  public class Stack<T> {
2
3      private final T[] data;
4      private int size;
5
6      @SuppressWarnings("unchecked")
7      public Stack(int capacity) {
8          data = (T[]) new Object[capacity];
9          size = 0;
10     }
11
12     public void push(T element) {
13         if (size == data.length) {
14             throw new RuntimeException("Stack overflow");
15         }
16         data[size] = element;
17         size++;
18     }
19
20     public T pop() {
21         if (size == 0) {
22             throw new RuntimeException("Stack underflow");
23         }
24         size--;
25         T element = data[size];
26         data[size] = null;
27         return element;
28     }
29
30     public T peek() {
31         if (size == 0) {
32             throw new RuntimeException("Stack is empty");
33         }
34         return data[size - 1];
35     }
36
37     public static void main(String[] args) {
38         Stack<Integer> stack = new Stack<>(10);
39         stack.push(1);
40         stack.push(2);
41         stack.push(3);
42         System.out.println(stack.pop());
43         System.out.println(stack.peek());
44         stack.push(4);
45         System.out.println(stack.pop());
46     }
47 }
48
```

Здесь при создании класса стека используется параметр типа `T`, который задаёт тип элементов, хранящихся в стеке. После чего инициализирую поля класса, первое – `data`, который является массивом элементов типа `T`, второе – текущая длина массива.

Дальше идет конструктор, перед которым стоит декоратор

`@SupressWarnings(«unchecked»)`, так как без него компилятор ругается при попытке кастинга массива `Object[]` к массиву `T[]`, кстати сам кастинг необходим из-за невозможности напрямую создавать массивы из дженерик типов.

После конструктора идёт определение основных методов. Первый метод для добавления элемента в стек. Если в стеке уже максимальное количество элементов – выбрасывается ошибка, если нет – элементу с индексом текущей длины присваивается значение переданного элемента, а сама длина увеличивается на 1.

Для метода `pop` реализована схожая проверка, только уже на пустоту стека. Если проверка пройдена, то от размера отнимается единица, элементу по индексу уменьшенного размера (по факту просто последний элемент стека) присваивается значение `null` (аналог удаления), и в конце возвращается значение удаленного элемента.

И последний метод `peek` для просмотра последнего элемента без его удаления. Такая же проверка, как и в предыдущем методе, единственное отличие – после этой проверки идёт одна строка с возвращением элемента с индексом `data.length - 1` (последний элемент)

Ну и метод `main` содержит в себе тестовые данные из методических материалов к данной лабораторной работе. Вывод ожидаемый:

3

2

4

В третьем задании необходимо было программу для учета продаж в магазине.

```
1  import java.util.Map;
2  import java.util.TreeMap;
3  import java.util.Scanner;
4
5  public class SalesTracker {
6
7      private TreeMap<String, Integer> sales = new TreeMap<>();
8
9      public void addSale(String product) {
10         sales.put(product, sales.getOrDefault(product, 0) + 1);
11     }
12
13     public void printSales() {
14         System.out.println("Products sold:");
15         for (Map.Entry<String, Integer> entry : sales.entrySet()) {
16             System.out.println(entry.getKey() + " - " + entry.getValue());
17         }
18     }
19
20     public int getTotalSales() {
21         int total = 0;
22         for (int count : sales.values()) {
23             total += count;
24         }
25         return total;
26     }
27
28     public String getTopProduct() {
29         String topProduct = null;
30         int maxCount = 0;
31         for (Map.Entry<String, Integer> entry : sales.entrySet()) {
32             if (entry.getValue() > maxCount) {
33                 maxCount = entry.getValue();
34                 topProduct = entry.getKey();
35             }
36         }
37         return topProduct;
38     }
39
40     public static void main(String[] args) {
41         SalesTracker tracker = new SalesTracker();
42         try (Scanner scanner = new Scanner(System.in)) {
43             System.out.println("Type product names to add sales. Press Enter on an empty line to finish");
44
45             while (true) {
46                 System.out.print("Product: ");
47                 String product = scanner.nextLine().trim();
48                 if (product.isEmpty()) {
49                     break;
50                 }
51
52                 tracker.addSale(product);
53             }
54
55             System.out.println();
56             tracker.printSales();
57             System.out.println("Total sales: " + tracker.getTotalSales());
58             System.out.println("Top product: " + tracker.getTopProduct());
59         }
60     }
61 }
62
```

В моем варианте необходимо было использовать TreeMap. Сначала я инициализирую сам словарь с продажами, где по ключу лежит товар, а по

значению – количество продаж. Далее я реализую основные методы. Первый это добавление в корзину, для его создания я использую простой метод put, в котором использую уже знакомую из первого задания конструкцию `getOrDefault + 1`.

Потом идет метод для вывода всех проданных продуктов и их количество(количество продаж). Для этого я использую тоже знакомое мне множество пар ключ-значение, в котором я циклом прохожусь по каждой паре и вывожу ее в консоль.

Для вывода общего количества продаж достаточно было пробежаться по значениям для каждого ключа и прибавить его к созданной в методе переменной `total`, изначально равной 0.

И наконец для последнего метода я все так же перебираю пары ключ-значение, только сравниваю их с `maxCount`, изначально равный 0.

В методе `main` я через цикл реализую ввод проданных продуктов, который завершится при вводе пустой строки. А в конце демонстрация всех методов созданного класса.

Вывод

В ходе лабораторной работы мной были освоены коллекции в языке Java.

Ссылка на гит - <https://github.com/BestStarProgrammer/IT-P/tree/main/lab2>